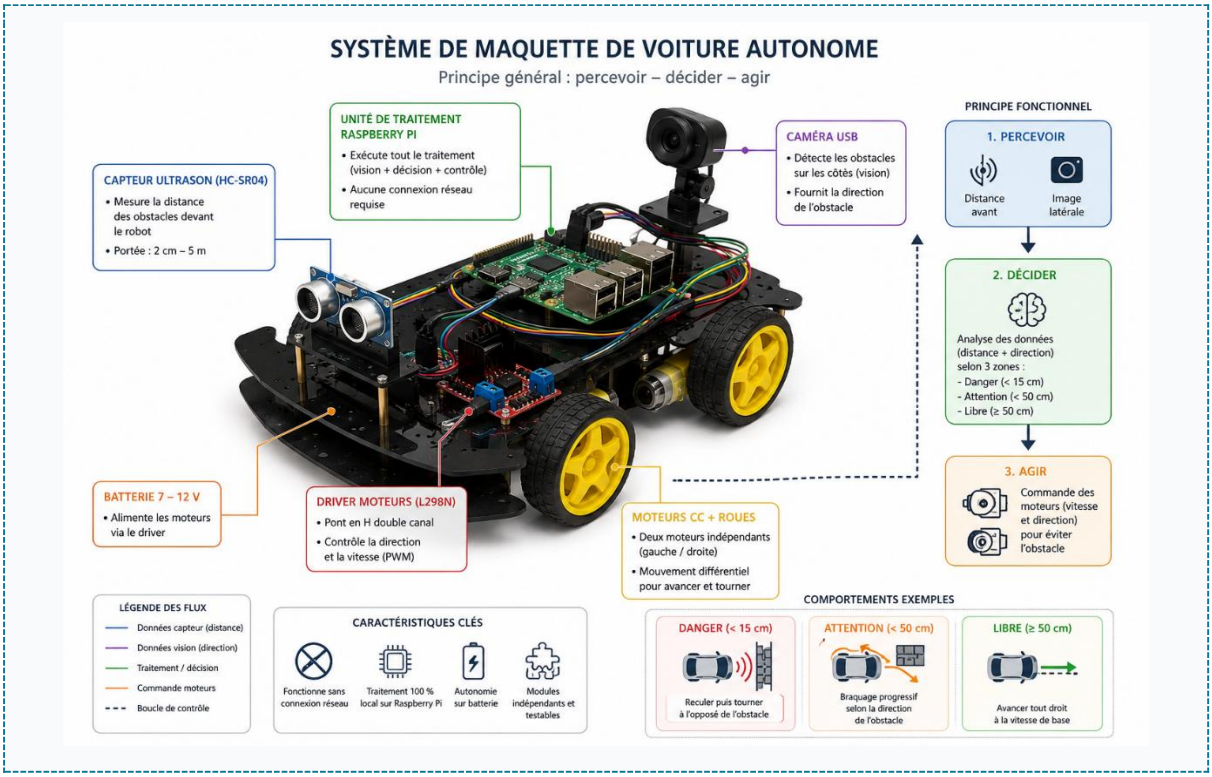
ROBOTICS PROJECT · RASPBERRY PI

Intelligent Control

Autonomous obstacle-avoidance robot

GOBERT Yanis

MARVIN Leo



Platform: Raspberry Pi | Language: Python | OpenCV | Hardware: HC-SR04

Table of Contents

1. System Overview

This project implements an **autonomous mobile robot** capable of detecting and avoiding obstacles in real time. It combines two complementary perception sources — an ultrasonic sensor that measures the **distance** to the obstacle, and a camera that estimates its **lateral position** — to drive two DC motors via an L298N H-bridge.

The logic is progressive: the robot drives straight ahead as long as the path is clear, adjusts its trajectory as soon as an obstacle approaches, and performs an emergency maneuver in case of immediate danger. All processing runs locally on the Raspberry Pi, with no GPU or network connection required.

Objectives

- **Perceive**: measure the distance to obstacles and detect their lateral position.
- **Decide**: choose the appropriate behavior based on the risk level.
- **Act**: drive both motors to avoid the obstacle without stopping.

2. Software Architecture

The code follows a clear split into four independent modules, organized along the **perception** → **decision** → **action** chain. This decoupling allows each part to be tested in isolation before integration.

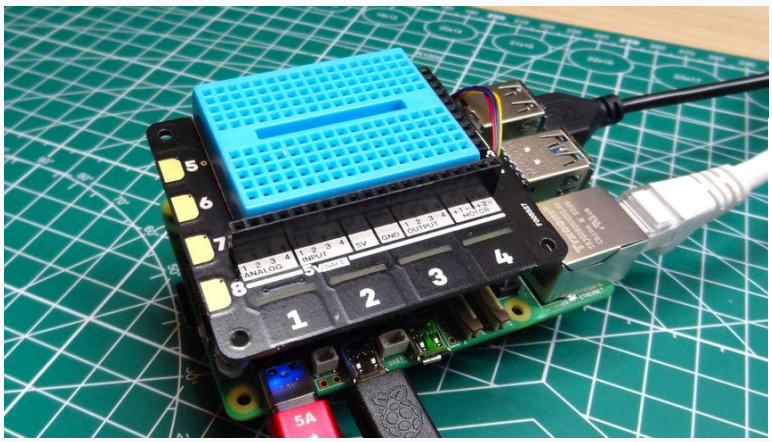
Module	Role	Key points
`ultrasonic.py`	Perception — distance	HC-SR04, time-of-flight, ~5 m range
`detector.py`	Perception — direction	USB camera + OpenCV, bias in [-1; +1]
`main.py`	Decision	Real-time loop, three-zone logic
`motors.py`	Action	L298N driver, differential PWM speeds

3. Required Hardware

The list below summarizes the components needed to build the robot.

Component	Role	Qty
`Raspberry Pi 4 / 5`	Brain: processing and control	**1**
`HC-SR04 sensor`	Ultrasonic distance measurement	**1**
`USB camera`	Lateral obstacle detection	**1**
`Driver L298N`	H-bridge: motor control	**1**
`DC motors + wheels`	Left/right propulsion	**2**
`Battery 7–12 V`	Motor power supply	**1**

Component	Role	Qty
Resistors 1 kΩ + 2 kΩ	Voltage divider on the ECHO line	**2**
Chassis, wires, breadboard	Structure and wiring	—



4. Wiring and Assembly

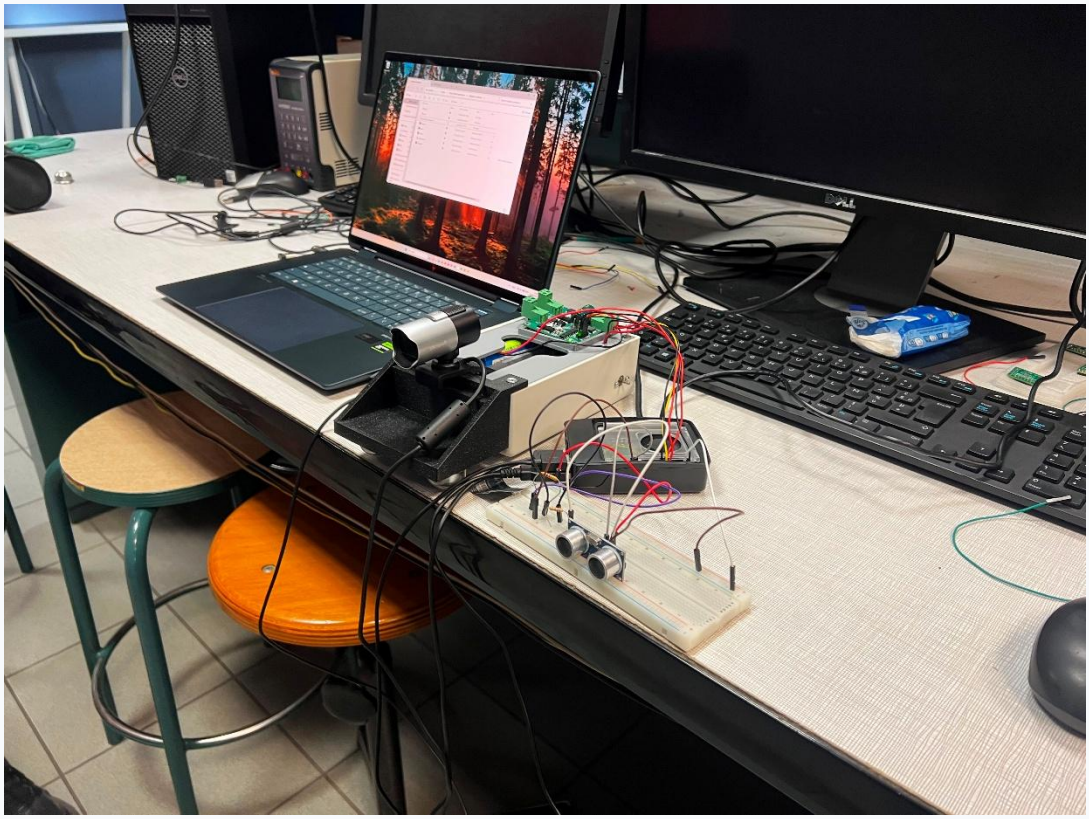
4.1 GPIO Pinout (BCM Numbering)

The table below details all the connections between the Raspberry Pi and the peripherals.

Pi Pin	Component	Function
GPIO 5	HC-SR04 TRIG	Triggers the pulse (direct connection)
GPIO 6	HC-SR04 ECHO	Echo return — via voltage divider
GPIO 17 / 27	L298N IN1 / IN2	Left motor direction
GPIO 22 / 23	L298N IN3 / IN4	Right motor direction
GPIO 18	L298N ENA	PWM speed — left motor
GPIO 24	L298N ENB	PWM speed — right motor
5 V / GND	HC-SR04 / L298N	Logic power and common ground

Three rules prevent most problems at this stage:

- **Common ground**: the GND of the Pi, the L298N, and the battery must all be connected together.
- **Separate power supplies**: the motors are powered by the battery via the L298N, never by the Pi.
- **Voltage adaptation**: only the ECHO line needs to be stepped down from 5 V to 3.3 V (see section 4.2).

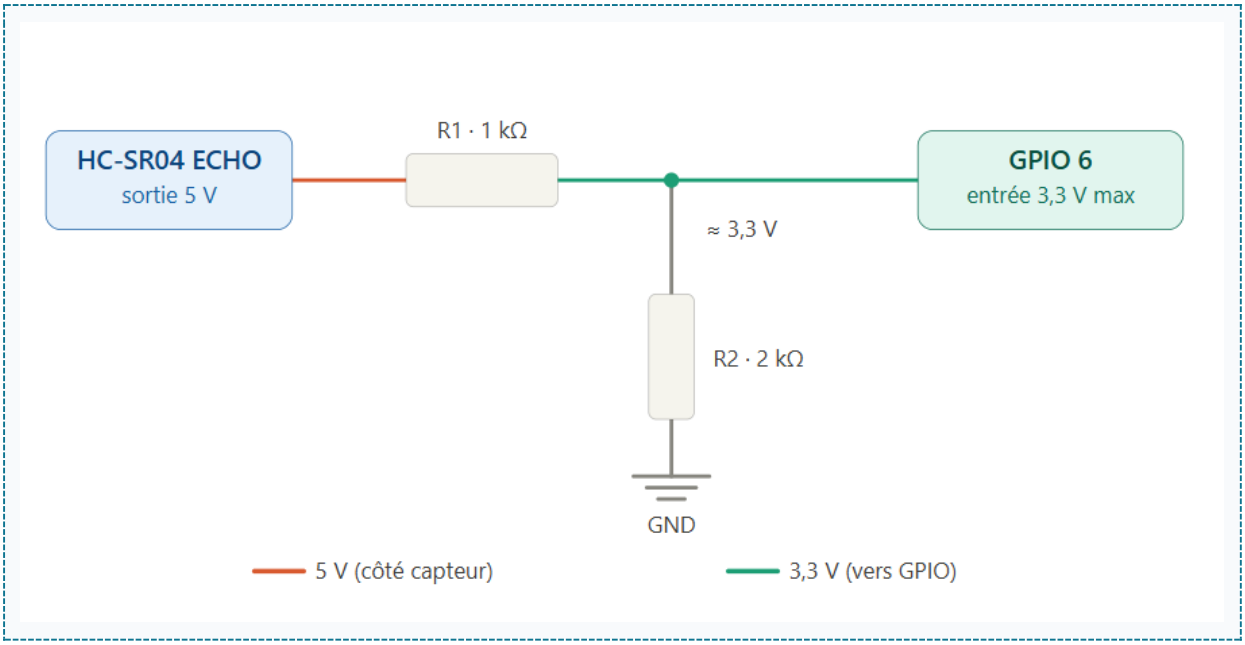


4.2 Voltage Divider on the ECHO Line

The ECHO pin of the HC-SR04 outputs a signal at **5 V**, while the Raspberry Pi GPIO inputs only tolerate **3.3 V maximum**. A voltage divider made of two resistors steps this voltage down:

1. Place **R1 (1 kΩ)** in series from the sensor's ECHO pin.
2. The other end of R1 forms the **midpoint**, from which the wire runs to **GPIO 6**.
3. Connect **R2 (2 kΩ)** between this midpoint and **ground**.

The voltage at the midpoint is then `V = 5 × R2 / (R1 + R2) = 5 × 2 / 3 ≈ 3.3 V`, just under the Pi's limit. Note that the TRIG pin, on the other hand, connects directly: the Pi sends 3.3 V, which the sensor accepts as is.



5. The L298N Driver: an H-Bridge

The L298N is a **dual H-bridge**: two identical, independent circuits, one per motor. Each bridge is made of four switches (transistors) surrounding the motor. Closing one diagonal pair makes the current flow in one direction; closing the other diagonal reverses it.

Direction, Speed, and Power

- **Direction**: the IN inputs set the direction. `IN1=HIGH, IN2=LOW` gives forward motion; the reverse combination gives reverse motion.
- **Speed**: a **PWM** signal on the ENA / ENB pins sets the speed. A 60% duty cycle corresponds to roughly 60% of maximum speed.
- **Power**: the motors draw their current from the battery; the L298N simply switches it according to the Pi's logic commands.

Two practical notes: the L298N **heats up** and causes a voltage drop of about 1.5 to 2 V (plan for a battery slightly above the motors' rated voltage), and it already includes **protection diodes** against voltage spikes from the windings.

6. Code Operation

6.1 Distance Measurement — ultrasonic.py

The sensor works by **time-of-flight**: it emits a pulse and measures how long it takes for the echo to return. The distance is derived from this duration and the speed of sound (343 m/s), divided by two for the round trip.

```
def distance_cm(self):
    GPIO.output(TRIG, GPIO.HIGH)
    time. sleep(0.00001)          # 10 us pulse
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == 0: # wait for echo
        start = time. time()
    while GPIO.input(ECHO) == 1: # echo duration
        end = time. time()
    return (end - start) * 34300 / 2   # cm
```

6.2 Visual Detection — detector.py

The camera indicates **where** the obstacle is. An adaptive background subtraction isolates the moving objects, the largest contour is kept, and its horizontal position is converted into a **bias** in [-1; +1]: negative on the left, positive on the right.

```
Mask = self. _bg. apply(band)        # background subtraction
contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cx = int(M['m10'] / M['m00'])     # contour center
bias = (cx - FRAME_W / 2) / (FRAME_W / 2)
```

6.3 Motor Control — motors.py

The sign of the speed determines the direction of rotation, and its absolute value sets the PWM duty cycle (i.e. the speed). The `MotorDriver` class initializes the pins and PWM itself when created.

```
def _set_motor (self, in1, in2, pwm, speed):
    speed = max (-100, min(100, speed))
    if speed >= 0:           # forward
        GPIO.output(in1, GPIO.HIGH)
        GPIO.output(in2, GPIO.LOW)
    else:                    # reverse
        GPIO.output(in1, GPIO.LOW)
        GPIO.output(in2, GPIO.HIGH)
    pwm.ChangeDutyCycle(abs(speed))
```

6.4 Decision Logic — main.py

The main loop chooses its behavior based on the measured distance, split into three zones:

Zone	Condition	Action
Danger	`distance < 15 cm`	Stop, reverse for 0.5 s, then turn 180° away from the obstacle
Warning	`distance < 50 cm`	Progressive avoidance: camera bias, proportional steering
Clear	`distance ≥ 50 cm`	Drive straight ahead at the base speed (60%)

The core of the steering logic is the `compute_wheel_speeds` function: the speed difference between the wheels combines the **direction** (bias) and the **urgency** (proximity). The closer the obstacle, the stronger the correction.

```
proximity = (WARNING_CM - distance) /
            (WARNING_CM - DANGER_CM)   # 0 -> 1
delta = bias * TURN_FACTOR * proximity
left  = BASE_SPEED + delta              # left wheel
right = BASE_SPEED - delta              # right wheel
```

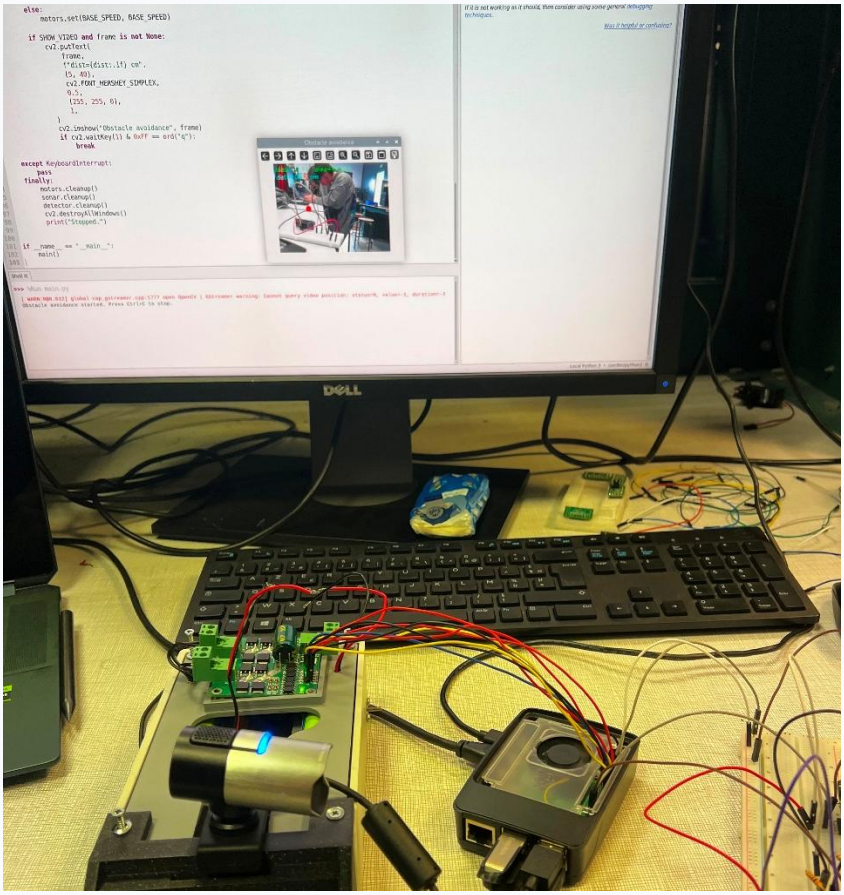
7. Setup and Commissioning

7.1 Testing Each Component Separately

Before running the full loop, test the sensor and motors separately. **Prop up the robot** (wheels off the ground) for the motor test.

- `test_ultrasonic.py`: continuously displays the distance — the value should follow your hand in front of the sensor.
- `test_motors.py`: spins each motor forward then backward — check the direction and the left/right correspondence.

If a motor spins backwards, simply **swap its two output wires** on the L298N terminal block — no code changes needed.



7.2 Running the Robot

Once the tests pass, run the full loop from the project folder:

```
cd robot
python3 main.py

# clean stop: Ctrl + C
```

On a Pi controlled headlessly (over SSH), set the line `SHOW_VIDEO = False` in `main.py`: otherwise the video display causes an error since there is no screen. The rest of the logic works fine in this mode.

8. Troubleshooting

Problems encountered and how they were resolved:

Symptom	Cause and solution
`Cannot determine SOC peripheral base address`	RPi.GPIO incompatible (Pi 5 / Bookworm). Install the replacement rpi-lgpio (no code changes needed).
The sensor always returns infinity	Check the voltage divider, the sensor's 5 V supply, and the common ground.
A motor spins backwards	Swap its two output wires on the L298N terminal block.
Motors erratic or unresponsive	Missing common ground between the Pi, the L298N, and the battery.
Motor whines without turning	Battery too weak, or voltage drop from the L298N: increase the supply voltage.

9. Conclusion and Future Work

The system demonstrates that robust obstacle avoidance can rely on simple, low-cost building blocks, provided their information is properly **combined** and the response properly **calibrated** to the level of risk. The modular architecture leaves plenty of room for future improvements.

Possible Improvements

- **PID control** on the steering to smooth the trajectory and avoid oscillations.
- **Learning-based detection** (embedded YOLO) to make obstacle recognition more reliable.
- **Multiple sensors**: several HC-SR04 units to widen the perception field.
- **Measurement filtering** (moving average) to reduce noise from the distance sensor.

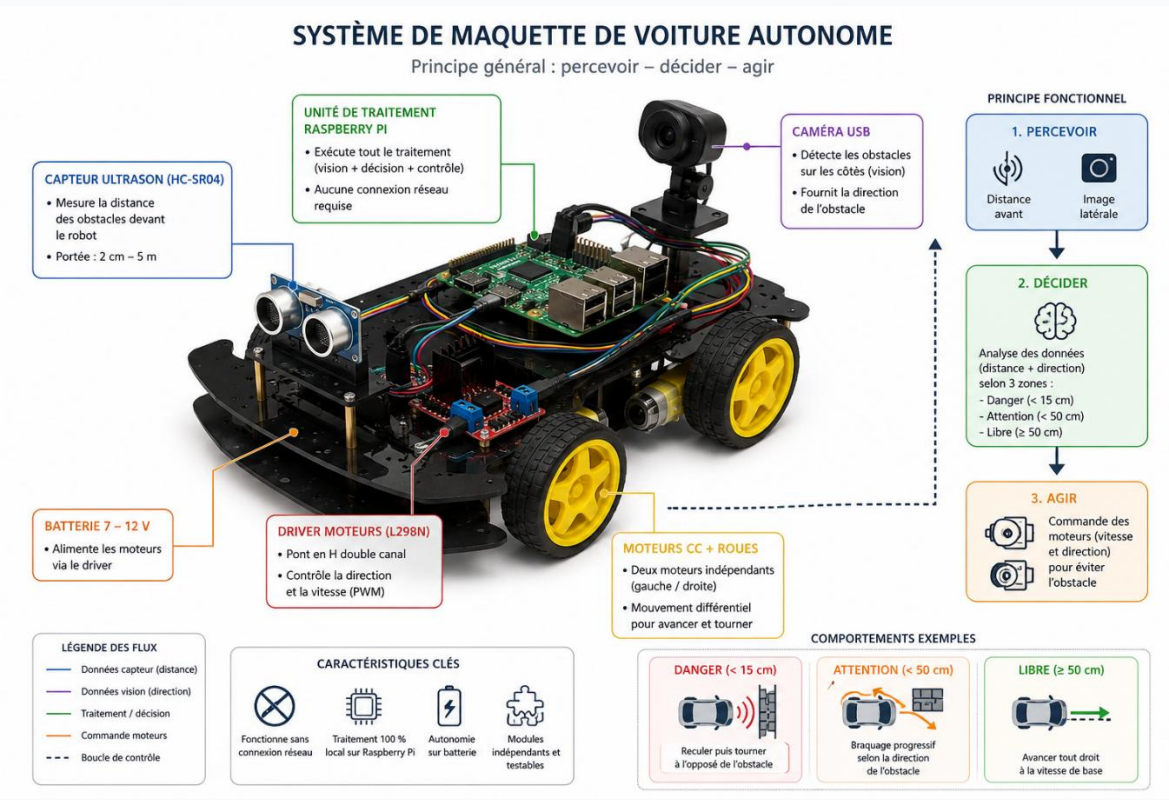


Diagram summarizing the operating principle of the prototype